

Overview

This portfolio summarizes selected programming work in Python, R, and Stata. The samples emphasize research data infrastructure, cloud-based data workflows, LLM-assisted classification, applied econometrics, panel data construction, and reproducible empirical analysis.

- **Python:** Flask services, BigQuery, Google Cloud Platform, Vertex AI/Gemini, API collection, web scraping, deduplication, classification tracking, resumable batch processing.
- **R:** High-frequency panel construction, joins across environmental datasets, Newey-West regressions, data visualization, recursive-additive aggregation for chain-weighted national accounts.
- **Stata:** Cross-country panel construction, PCA index construction, fixed effects, Difference-GMM, System-GMM, VAR, unit-root testing, ARCH/GARCH, automated figures and reports.

Sample 1: K-12 AI Use Case Research Infrastructure

Language and tools: Python, Flask, SQL, BigQuery, Google Cloud Platform, Vertex AI/Gemini, Cloud Run, requests, BeautifulSoup, trafilatura.

Purpose. This research assistant project builds infrastructure to identify and classify AI use cases and institutional AI adoption in U.S. public K-12 education from public news and institutional sources. The system is designed as a research pipeline rather than a one-time classifier: it collects source material, stores raw articles, deduplicates records, applies LLM-based triage and extraction, records audit data, and exports validation samples.

Representative contributions.

- Built and maintained Flask-based services for raw article collection and classification.
- Collected data from NewsAPI, SerpApi, GNews, GDELT, RSS feeds, and web pages.
- Parsed and stored full text, metadata, stable article IDs, and raw JSON in BigQuery.
- Implemented a two-stage Gemini workflow: lightweight triage followed by structured extraction for retained records.
- Added manual-review thresholds, audit rows, cleanup endpoints, deduplication logic, health checks, and BigQuery-backed progress tracking.

Representative code excerpt: robust article ID and audit-row design.

```
def make_article_id(url: str) -> str:
    return hashlib.sha1((url or '').strip().encode('utf-8')).hexdigest()

def _build_triage_audit_row(article: Dict[str, Any], *, status: str,
                           article_type: str = "",
                           relevance_score: Optional[int] = None,
                           manual_review: Optional[bool] = None,
                           reason: str = "",
                           error_message: str = "",
                           classified_at: Optional[str] = None,
                           triage_flags: Optional[Dict[str, Optional[bool]]] = None) -> Dict[str, Any]:
    url = str(article.get('url', '') or '').strip()
    article_id = make_article_id(url)
    classified_at = classified_at or datetime.utcnow().isoformat()
    triage_flags = triage_flags or {field: None for field in TRIAGE_FLAG_FIELDS}

    payload = "|".join([
        article_id, classified_at, status, str(article_type or ""),
        "" if relevance_score is None else str(relevance_score),
        reason or "", error_message or "",
    ])
    audit_id = hashlib.sha1(payload.encode('utf-8')).hexdigest()

    return {
        "audit_id": audit_id,
        "article_id": article_id,
        "classified_at": classified_at,
```

```

    "status": status,
    "article_type": article_type or "",
    "relevance_score": relevance_score,
    "manual_review": manual_review,
    "reason": reason or "",
    "title": str(article.get('title', '')) or '',
    "url": url,
    **{field: triage_flags.get(field) for field in TRIAGE_FLAG_FIELDS},
}

```

Representative code excerpt: BigQuery-backed resumable batch runner.

```

@dataclass
class BatchResult:
    batch_index: int
    article_count: int
    response_status: int
    classified: int
    filtered_out: int
    errors: int
    retries: int
    elapsed_seconds: float
    ok: bool
    message: str
    payload: Dict[str, Any]

def iter_chunks(values: List[str], size: int) -> Iterable[List[str]]:
    for i in range(0, len(values), size):
        yield values[i:i + size]

def get_token(force: bool = False) -> str:
    with TOKEN_LOCK:
        now = time.time()
        token = TOKEN_CACHE["value"]
        ts = float(TOKEN_CACHE["ts"])
        if not force and token and (now - ts) < TOKEN_REFRESH_SECONDS:
            return str(token)
        output = subprocess.check_output(
            ["Users/joelpark/google-cloud-sdk/bin/gcloud",
             "auth", "print-identity-token"],
            text=True,
        ).strip()
        TOKEN_CACHE["value"] = output
        TOKEN_CACHE["ts"] = now
        return output

```

Sample 2: Air Pollution and Solar PV Generation

Language and tools: R, dplyr, stringr, lubridate, ggplot2, sandwich, lmtest, psych, broom, haven, readr, tidyr.

Purpose. This co-authored journal article studies how particulate matter affects solar PV generation in Chungcheongnam-do, Korea. My R workflow constructs an hourly and daily panel by merging air pollution, meteorological, wind-direction, and solar generation data.

Representative contributions.

- Cleaned Korean monitor names and addresses and extracted city identifiers.
- Created hourly datetime variables from multiple possible raw date formats.
- Joined PM10/PM2.5, weather, and PV generation data by city and time.
- Constructed hourly and daily panels, wind-direction bins, and descriptive statistics.
- Estimated baseline and wind-interaction regressions with Newey-West standard errors.

```

pm_weather <- pm %>%
  filter(!is.na(city), city != "", !is.na(datetime)) %>%
  select(monitor_name, address, datetime, all_of(pm_vars), city) %>%
  inner_join(weather_for_join, by = c("city", "datetime"))

pm_province_hourly <- pm_weather %>%
  group_by(datetime) %>%
  summarise(
    PM10 = mean(PM10, na.rm = TRUE),
    PM25 = mean(PM25, na.rm = TRUE),
    temp = mean(temp, na.rm = TRUE),
    ws = mean(ws, na.rm = TRUE),

```

```

humid = mean(humid, na.rm = TRUE),
cloud_cover = mean(cloud_cover, na.rm = TRUE),
solar_irradiance = mean(solar_irradiance, na.rm = TRUE),
sunshine_duration = mean(sunshine_duration, na.rm = TRUE),
u = mean(ws * cos(wd * pi / 180), na.rm = TRUE),
v = mean(ws * sin(wd * pi / 180), na.rm = TRUE),
.groups = "drop"
) %>%
mutate(wd_mean = (atan2(v, u) * 180 / pi + 360) %% 360)

br25 <- lm(
  solar ~ PM25 + temp + humid + cloud_cover +
    solar_irradiance + sunshine_duration +
    factor(hour) + factor(month),
  data = panel_hourly
)
br25_nw <- coeftest(br25, vcov = NeweyWest(br25, lag = 24))

```

Sample 3: Recursive-Additive Closure for Chain-Weighted Growth Rates

Language and tools: R, dplyr, tidyr, purrr, readr, ggplot2, haven.

Purpose. This paper addresses non-additivity in chain-weighted national accounts. The R workflow diagnoses the gap between published chained real levels and the sum of component real levels, then constructs recursive-additive component series from official growth rates and base-year nominal shares.

Representative contributions.

- Loaded and cleaned industry-level national accounts data.
- Computed published chained aggregate growth and component growth primitives.
- Built a recursive-additive aggregator that works forward and backward from a base year.
- Produced base-year robustness tables and figure-ready outputs.

```

build_ra <- function(baseyear, df, GDP_ID = 1, NOMVAR = "nva") {
  df <- df %>% arrange(nind, year)

  gdp0 <- df %>%
    filter(nind == GDP_ID) %>%
    arrange(year) %>%
    transmute(year, Y_ch = rva, g_ch = log(rva) - log(lag(rva)))

  comp0 <- df %>%
    filter(nind != GDP_ID) %>%
    group_by(nind) %>%
    arrange(year, .by_group = TRUE) %>%
    mutate(Y_ch_i = rva, g_i = log(rva) - log(lag(rva))) %>%
    ungroup()

  base_w <- df %>%
    filter(year == baseyear, nind != GDP_ID) %>%
    transmute(nind, nva = .data[[NOMVAR]]) %>%
    mutate(w = nva / sum(nva, na.rm = TRUE)) %>%
    select(nind, w)

  Y0 <- gdp0 %>% filter(year == baseyear) %>% pull(Y_ch)
  # initialize component real levels with base-year nominal shares,
  # then recursively apply official component growth rates
}

```

Sample 4: Honors Thesis Dynamic Panel Workflow

Language and tools: Stata, xtabond2, PCA, fixed effects, Difference-GMM, System-GMM, automated graph/table generation.

Purpose. My honors thesis, *The Uneven Growth Cost of Political Instability*, studies the heterogeneous growth effects of political instability using a global panel of 185 countries from 1960 to 2023.

Representative contributions.

- Built a Stata workflow for data loading, variable construction, and panel setup.

- Constructed GDP per capita, population growth, investment, trade, government expenditure, human capital, and TFP variables.
- Filled and interpolated education variables across multiple data vintages.
- Constructed political instability indices using PCA.
- Estimated pooled OLS, fixed effects, Difference-GMM, and System-GMM models with income-group heterogeneity.

```
capture program drop prog_gmm_het
program define prog_gmm_het
    syntax , INSTAB(name) NAME(name) [RHS(string) EXTRA_IV(string)]

    if "`rhs'" == "" local rhs "$SCORE $ADD_FULL"

    quietly xtabond2 dlny L.lny `rhs' ///
        `instab'_low `instab'_middle `instab'_high i.period5 ///
        if !missing(income_grp), ///
        gmm(lny, lag(2 3) collapse) ///
        gmm(`instab',lag(1 2) collapse) ///
        iv(i.period5 `rhs' low middle high `extra_iv') ///
        twostep robust small
    estimates store `name'
end

foreach nm of local pca_names {
    pca `nm'_vars', components(1) corr
    predict double _tmp_s, score
    egen double `nm' = std(_tmp_s)
    drop _tmp_s
}
```

Sample 5: Time-Series Econometrics in Stata

Language and tools: Stata, ARMA, ARCH/GARCH, VAR, unit-root tests, lag-order selection, seasonal dummies, cross-correlation, automated graph export.

Purpose. In graduate time-series coursework, I wrote Stata scripts for model selection, diagnostic testing, and presentation-ready figures. One project estimated inflation and policy uncertainty VAR specifications; another evaluated ARMA/ARCH/GARCH models for sales, visits, output, and input variables.

```
tsset mdate, monthly

gen cpi_hat = .
replace cpi_hat = 100 in 1
replace cpi_hat = L.cpi_hat*(1 + infl_mom/100) if _n > 1
gen infl_yoy = 100*((cpi_hat/L12.cpi_hat) - 1)

varsoc infl_mom fepu if !missing(infl_mom), maxlag(12)

gen monthnum = month(dofm(mdate))
quietly tab monthnum, gen(mdum)

var infl_mom fepu if !missing(infl_mom), ///
    lags(1/4) exog(mdum2-mdum12)

irf create baseline, replace
irf graph oirf, impulse(feup) response(infl_mom)
```

Sample 6: Applied Econometrics and Reproducible Reports

Language and tools: Stata dynamic documents, R Markdown, tidyverse, infer, broom, modelr.

Purpose. Coursework projects demonstrate reproducible analysis and communication: Stata dynamic reports for regression analysis and omitted-variable-bias exercises, and R Markdown reports for statistical inference and visualization.

Representative examples.

- Built Stata dynamic reports with commands and interpretations embedded together.
- Used robust standard errors and regression tables for applied microeconomics problem sets.

- Used R Markdown, tidyverse, infer, modelr, and broom for data cleaning, visualization, hypothesis testing, and report generation.

```
college_reduced <- college %>%
  select(INSTNM, PCIP11, PCIP14, PCIP15, PCIP26, PCIP27,
         PCIP40, PCIP41, PCTFLOAN) %>%
  rename(
    college_name = INSTNM,
    comp_sci = PCIP11,
    engineering = PCIP14,
    pct_fed_loan = PCTFLOAN
  ) %>%
  mutate(
    stem_pct_total = rowSums(
      select(., comp_sci, engineering, eng_tech,
             biology, math, physical_sci, science_tech),
      na.rm = TRUE
    ),
    stem_dominant = if_else(
      stem_pct_total > mean(stem_pct_total, na.rm = TRUE),
      "STEM Dominant",
      "Non-STEM Dominant"
    )
  )

obs_diff <- college_inference %>%
  specify(pct_fed_loan ~ stem_dominant) %>%
  calculate(stat = "diff in means",
           order = c("STEM Dominant", "Non-STEM Dominant"))
```

Technical Summary

- **Production-style research infrastructure:** Flask services, API ingestion, full-text parsing, deduplication, BigQuery storage, Cloud Run deployment patterns, health endpoints, batch processing, and audit tables.
- **AI/ML workflows:** Vertex AI/Gemini prompts, two-stage classification/extraction, manual-review thresholds, validation exports, error handling, and model-output auditing.
- **Data engineering:** Stable IDs, schema design, table partitioning/clustering concepts, long-running batch progress tracking, CSV/DTA/RDS ingestion, and panel construction.
- **Applied econometrics:** Fixed effects, robust standard errors, Newey-West corrections, PCA, ARMA, ARCH/GARCH, VAR, Difference-GMM, System-GMM, and growth-accounting workflows.
- **Reproducibility:** Stata do-files, R scripts, R Markdown reports, LaTeX writing, documented code structure, and figure/table export.